

Experiment No: 04

Date: 17/09/2026

Title: UPI Payment System using Interface and Exception Handling

Objective / Aim

To design and implement a UPI Payment System in Java using interfaces, classes, encapsulation, custom exceptions, try-catch-finally, and method overriding.

System Requirements

Hardware

- Standard PC/Laptop
- Intel Core i3/i5 processor or equivalent
- Minimum 4 GB RAM
- Minimum 500 MB free storage

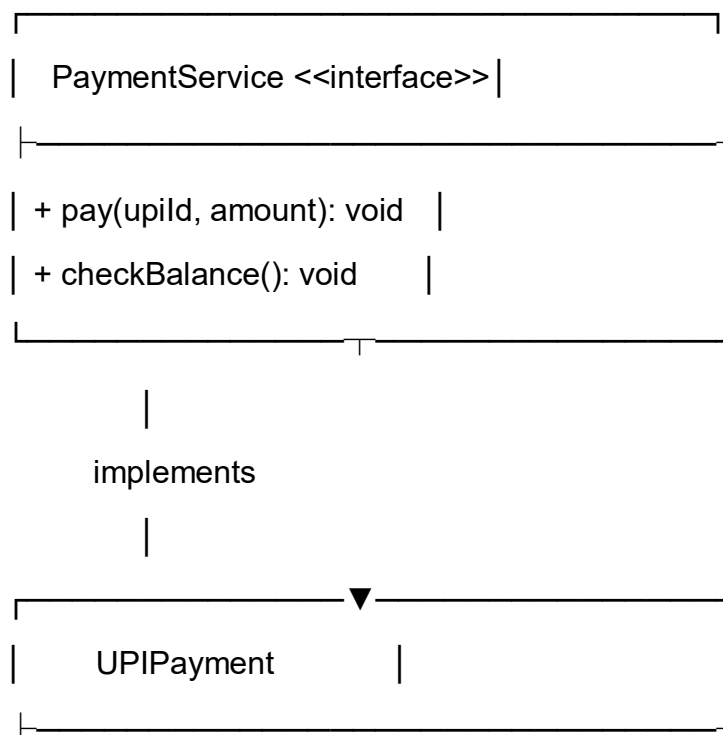
Software

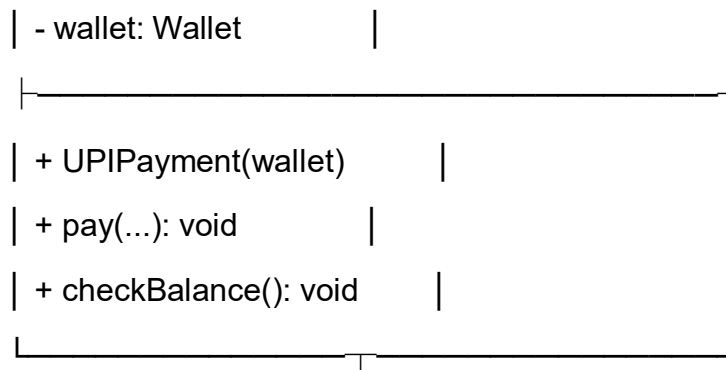
- Operating System: Windows / Linux / macOS
- Java Development Kit (JDK 17 or JDK 21)
- Text Editor or IDE such as Eclipse, IntelliJ IDEA, NetBeans, or VS Code
- Command Prompt / Terminal

⚙️ Algorithm / Logic Description

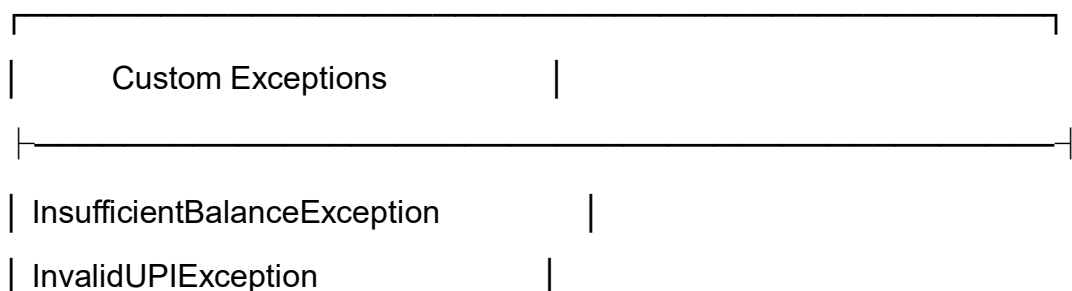
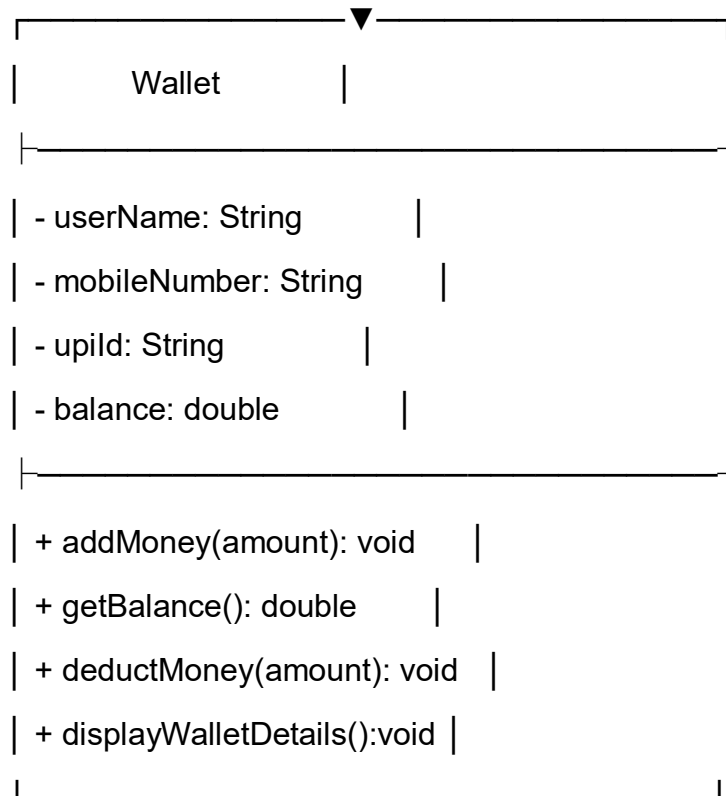
1. Create a "PaymentService" interface with payment and balance methods.
2. Create custom exceptions for invalid UPI ID, invalid amount, and insufficient balance.
3. Create a "Wallet" class to store user and balance details.
4. Define methods to add, deduct, and check wallet balance.
5. Create a "UPIPayment" class that implements the "PaymentService" interface.
6. Validate the UPI ID and payment amount.
7. Check whether sufficient wallet balance is available.
8. Deduct the payment amount if all conditions are valid.
9. Handle errors using "try-catch" blocks.
10. Use "finally" to display the transaction completion message.

📐 UML Class Diagram





uses



InvalidAmountException	

Relationship: "UPIPayment" implements the "PaymentService" interface and uses the "Wallet" class. The custom exception classes are used to handle different payment errors.

Source Code

```
package javacore;
```

```
public class MainUPIPayment {
```

```
    // PaymentService Interface
```

```
    interface PaymentService {
```

```
        void pay(String upild, double amount);
```

```
        void checkBalance();
```

```
    }
```

```
    // Custom Exception for insufficient balance
```

```
    static class InsufficientBalanceException extends Exception {
```

```
        public InsufficientBalanceException(String message) {
```

```
            super(message);
```

```
        }
```

```
}
```

```
// Custom Exception for invalid UPI ID
```

```
static class InvalidUPIException extends Exception {
```

```
    public InvalidUPIException(String message) {
```

```
        super(message);
```

```
    }
```

```
}
```

```
// Custom Exception for invalid amount
```

```
static class InvalidAmountException extends Exception {
```

```
    public InvalidAmountException(String message) {
```

```
        super(message);
```

```
    }
```

```
}
```

```
// Wallet Class
```

```
static class Wallet {
```

```
    // Private data members
```

```
    private String userName;
```

```
    private String mobileNumber;
```

```
    private String upid;
```

```
    private double balance;
```

```
// Constructor
public Wallet(String userName, String mobileNumber,
               String upild) {

    this.userName = userName;
    this.mobileNumber = mobileNumber;
    this.upild = upild;
    this.balance = 0.0;
}

// Method to add money
public void addMoney(double amount) {

    if (amount > 0) {
        balance = balance + amount;
        System.out.println(
            "Money added successfully: ₹" + amount
        );
    } else {
        System.out.println("Invalid amount.");
    }
}

// Method to get balance
public double getBalance() {
    return balance;
}
```

```

// Method to deduct money
public void deductMoney(double amount) {
    balance = balance - amount;
}

// Method to display wallet details
public void displayWalletDetails() {

    System.out.println("\n----- Wallet Details -----");
    System.out.println("User Name    : " + userName);
    System.out.println("Mobile Number: " + mobileNumber);
    System.out.println("UPI ID      : " + upid);
    System.out.println("Balance     : ₹" + balance);
}
}

```

```

// UPI Payment Class
static class UPIPayment implements PaymentService {

```

```

    private Wallet wallet;

```

```

// Constructor

```

```

public UPIPayment(Wallet wallet) {
    this.wallet = wallet;
}

```

```

// Pay method

```

```

@Override

```

```
public void pay(String upid, double amount) {

    try {

        // Validate UPI ID
        if (upid == null ||
            !upid.contains("@") ||
            upid.startsWith("@") ||
            upid.endsWith("@")) {

            throw new InvalidUPIException(
                "Invalid UPI ID: " + upid
            );
        }

        // Validate payment amount
        if (amount <= 0) {

            throw new InvalidAmountException(
                "Payment amount must be greater than zero."
            );
        }

        // Check sufficient balance
        if (amount > wallet.getBalance()) {

            throw new InsufficientBalanceException(
                "Insufficient balance for this payment."
            );
        }
    }
}
```

```
}

// Deduct payment amount
wallet.deductMoney(amount);

System.out.println("\nPayment Successful!");
System.out.println("Receiver UPI ID: " + upid);
System.out.println("Amount Paid    : ₹" + amount);

}

catch (InvalidUPIException e) {

    System.out.println(
        "Transaction Failed: " + e.getMessage()
    );
}

catch (InvalidAmountException e) {

    System.out.println(
        "Transaction Failed: " + e.getMessage()
    );
}

catch (InsufficientBalanceException e) {

    System.out.println(
        "Transaction Failed: " + e.getMessage()
    )
}
```

```

        );
    }

    finally {

        System.out.println(
            "Transaction processing completed."
        );
    }
}

// Check balance method
@Override
public void checkBalance() {

    System.out.println(
        "Available Wallet Balance: ₹"
        + wallet.getBalance()
    );
}
}
}
}

```

Compilation & Execution Commands

Save the source code as:

MainUIPayment.java

If you are using the "package javacore;" statement, save the file inside the appropriate "javacore" package/folder.

Compile the program:

```
javac MainUPIPayment.java
```

Run the program:

```
java MainUPIPayment
```

«**Note:** The code you provided does not contain a "main()" method inside "MainUPIPayment", so it will compile but will not produce output when run. A "main()" method is required to create the "Wallet" and "UPIPayment" objects and perform transactions.»

Output Screen

Example 1 – Successful Payment

Money added successfully: ₹5000.0

Payment Successful!

Receiver UPI ID: abc@upi

Amount Paid : ₹1000.0

Transaction processing completed.

Available Wallet Balance: ₹4000.0

Example 2 – Invalid UPI ID

Transaction Failed: Invalid UPI ID: @upi

Transaction processing completed.

Example 3 – Invalid Amount

Transaction Failed: Payment amount must be greater than zero.

Transaction processing completed.

Example 4 – Insufficient Balance

Transaction Failed: Insufficient balance for this payment.

Transaction processing completed.

Result

The UPI Payment System was successfully designed using Java. The program demonstrates interfaces, encapsulation, custom exceptions, exception handling, method overriding, and the "finally" block for processing UPI payments and handling transaction errors.